

LearnMate

AI-Powered Learning Management System & Real-Time Collaboration Platform

Master Technical Documentation, System Blueprint, API Reference & Developer Onboarding Manual

Version: 2.0.0 (Production Release)	Architecture: Decoupled Microservices & ASGI
Backend Core: Django 6.0 + DRF + Daphne	Frontend SPA: React 19 + Vite 8 + Tailwind v4
AI / RAG Service: FastAPI + ChromaDB + Groq	Async / Queues: Celery 5.4 + Redis 6379
Media Cloud: AWS S3 (Presigned URLs)	Database: PostgreSQL 16 / SQLite 3
Real-Time: Django Channels 4.0 (WebSockets)	Target Audience: Full-Stack Engineers & DevOps

DOCUMENT SCOPE & SYSTEM GOALS

Authoritative Engineering Blueprint:
This document is the exhaustive technical specification for **LearnMate**. It details the end-to-end architecture, database schema, authentication & MFA pipelines, REST API endpoints, bidirectional WebSocket communication, the RAG AI Tutoring engine (FastAPI, ChromaDB, Groq LLM, automated Whisper/YouTube speech transcription), Celery background tasks, AWS S3 cloud security, production Nginx/SSL deployment, and a complete **Developer Onboarding Guide** for rapid engineering orientation.

TABLE OF CONTENTS	SECTION
1. Executive Project Overview & Business Problem	Section 1
2. Implemented Features vs Planned Roadmap	Section 2
3. Technology Stack & Architectural Specifications	Section 3
4. End-to-End System Architecture & Service Topology	Section 4
5. Complete Project Directory Structure & File Manifest	Section 5
6. Database Architecture & Relational ER Schema	Section 6
7. Authentication, Multi-Factor Auth (MFA/TOTP) & Security	Section 7
8. Backend Core Architecture (Django 6 & DRF)	Section 8
9. AI Microservice & RAG Tutoring Pipeline (FastAPI + ChromaDB)	Section 9
10. Speech Transcription, Translation & Vector Embeddings	Section 10
11. Real-time WebSockets Messaging (Channels + Redis)	Section 11
12. Asynchronous Workers & Periodic Scheduling (Celery + Redis)	Section 12
13. Frontend Architecture & Client State (React 19 + Vite)	Section 13
14. Cloud Media Storage & AWS S3 Signed URL Architecture	Section 14
15. Comprehensive HTTP REST API Reference	Section 15
16. Environment Configuration & Secret Management	Section 16
17. Production Deployment, Nginx, Systemd, Domain & SSL	Section 17
18. Testing, Verification, QA & Health Checks	Section 18
19. Troubleshooting Manual & Common Failure Modes	Section 19
20. Developer Onboarding: Rapid Codebase Bootstrap Guide	Section 20

1. Executive Project Overview & Business Value

LearnMate is a production-grade, multi-role Learning Management System (LMS) and collaborative real-time ecosystem engineered to bridge the communication and learning gap between **Students**, **Mentors**, and **Platform Administrators**. Unlike traditional static learning platforms, LearnMate blends core course authoring and video delivery with cutting-edge **Retrieval-Augmented Generation (RAG) AI Tutoring** grounded directly in lesson lecture transcripts, high-throughput bidirectional WebSockets chat, and automated asynchronous learner engagement workflows.

Key Business Problems Solved:

- **Eliminate Passive Learning Isolation:** Embeds an AI Tutor directly inside the video lesson player that answers questions with strict factual grounding in the exact lecture content.
- **Frictionless Multilingual Accessibility:** Automatically extracts audio from YouTube lectures or uploaded video files, transcribes speech with Faster-Whisper, detects non-English languages, and translates the transcript to English via Groq LLMs before vectorization.
- **Real-Time Mentor-Student Connectivity:** Live 1-on-1 direct messaging and batch cohort discussions running on persistent ASGI WebSockets with typing indicators and online presence tracking.
- **Automated Retention & Inactivity Nudges:** Automated Celery Beat background jobs track student progress, detect inactivity beyond 72 hours, and dispatch personalized email nudges via Brevo SMTP.
- **Enterprise Security & Media Protection:** Zero unauthenticated direct access to lesson videos or PDFs. AWS S3 private storage with time-limited presigned URLs (1-hour expiration) combined with JWT authentication and RFC 6238 TOTP Multi-Factor Authentication.

2. Implemented Features vs Planned Roadmap

Domain / Module	Implemented & Verified Features (Live in v2.0)	Status
Student Portal	• Course discovery catalog with search and level filters • Single-click enrollment with progress tracker • Custom video lesson player with module navigation sidebar • Contextual AI Tutor slide-out drawer grounded in lesson transcripts • PDF/Document resource downloader with AWS S3 signed URLs • Real-time 1-on-1 chat with assigned mentors and cohort group chats • Continue Learning quick-resume shortcut and completion certificate emails	LIVE
Mentor Studio	• Course creator studio (Title, Description, Level, Thumbnail) • Multi-level syllabus editor: Course → Modules → Lessons → Resources • Video ingest: YouTube URL parsing or direct video file upload to AWS S3 • Automated background transcription (Whisper) & RAG vectorization • Student enrollment rosters & granular lesson-by-lesson progress tracking • Cohort analytics & direct messaging with enrolled students	LIVE
Admin Control	• Global analytical dashboard (User counts, Course counts, Enrollments) • User moderation: Block/unblock students and mentors, role promotion • Course publication review workflow: Approve (Publish), Reject (Draft) • System-wide analytical reports and audit logs	LIVE
AI & RAG Engine	• Dedicated FastAPI microservice on Port 8001 with shared secret auth • Word-level sliding window chunker (400 words, 50-word overlap) • SentenceTransformers <i>all-MiniLM-L6-v2</i> dense vector embeddings • ChromaDB vector store with metadata filtering by <i>course_id</i> • Groq <i>openai/gpt-oss-120b</i> high-speed LLM generation with strict course grounding • Multilingual Whisper transcription + Groq translation pipeline	LIVE
Real-time Chat	• Django Channels 4.0 ASGI WebSocket consumers with JWT authentication • Redis Channel Layer for pub/sub message distribution • Direct 1-on-1 chat rooms with read receipts & typing indicators • Batch cohort group messaging with persistent history	LIVE
Planned Roadmap	• Interactive Quiz and Auto-Graded Assignment Engine • Stripe / Razorpay Payment Gateway integration for paid premium courses • WebRTC Live 1-on-1 video mentoring sessions & screen sharing • Gamification badges, leaderboards, and PDF certificate generation	PLANNED

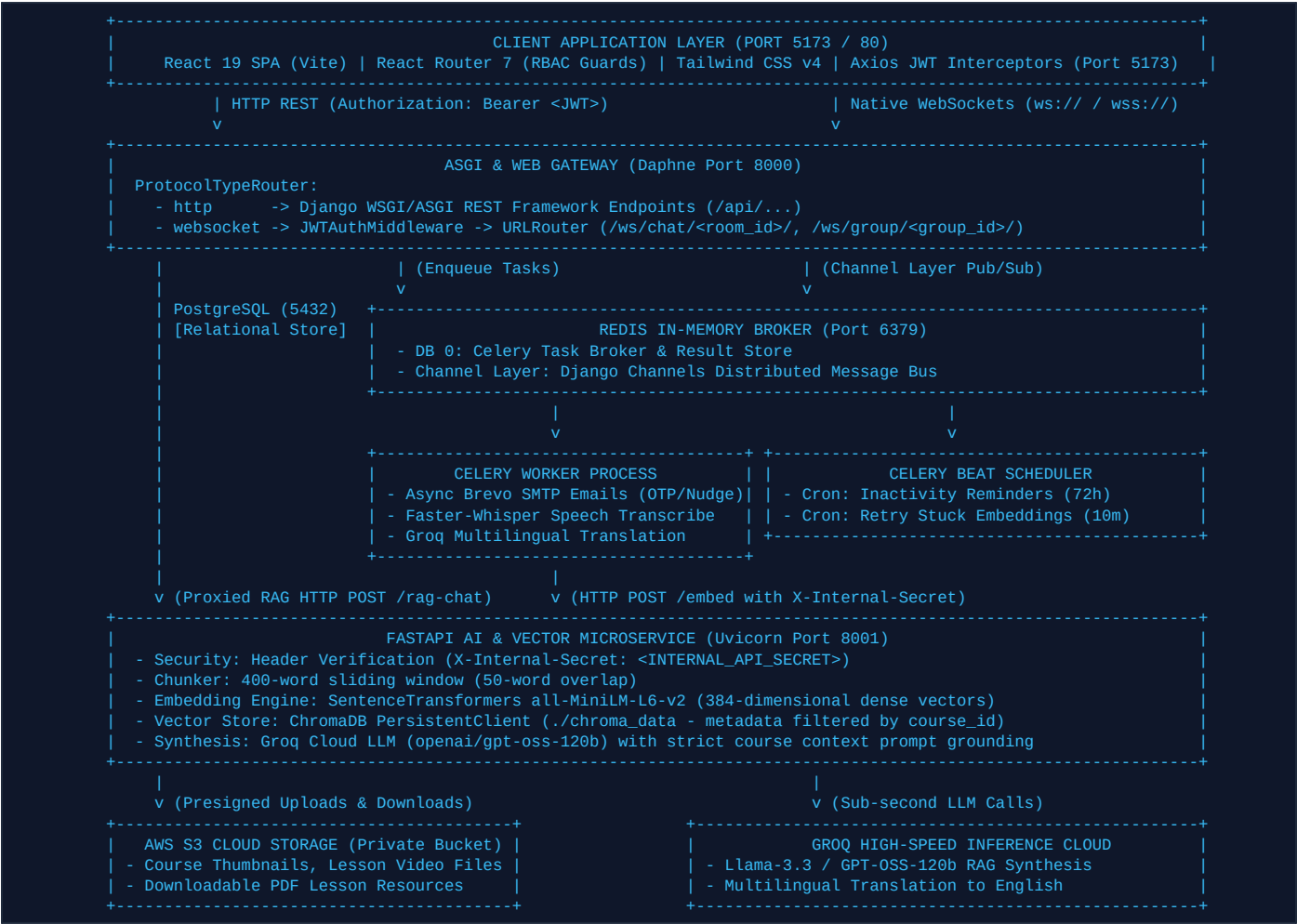
3. Technology Stack & Architectural Specifications

The following table enumerates all production technologies, exact versions, and architectural responsibilities:

Component / Layer	Technology	Version	Architectural Responsibility
Frontend SPA	React 19	19.2.0	Component hierarchy, hooks, state reactivity, UI rendering
Build Tooling	Vite	8.0.0	Fast ESM dev server, optimized production bundling
Client Routing	React Router DOM	7.13.0	Declarative routing & Role-based route guards (RBAC)
Styling	Tailwind CSS	v4.0.0	Modern utility design, responsive grid, dark/light themes
HTTP Client	Axios	1.18.0	REST API client with 401 JWT token auto-refresh interceptor
Backend Web Core	Python & Django	3.10+ / 6.0.5	Relational ORM, business logic, settings, security middleware
REST API Engine	Django REST Framework	3.17.0	Model serializers, permission classes, viewsets
Auth & Tokens	SimpleJWT + PyOTP	5.3+ / 2.9+	HMAC-SHA256 JWT tokens, OTP verification, RFC 6238 TOTP MFA
ASGI / WebSockets	Daphne & Channels	4.1+ / 4.2+	Async protocol router, WebSocket consumer lifecycles
Channel Layer	Redis + channels_redis	7.x / 4.2+	Distributed WebSocket pub/sub message broker & room groups
Task Queue	Celery & Celery Beat	5.4.0	Non-blocking background email dispatch, Whisper transcription, cron
AI Service	FastAPI & Uvicorn	0.115+ / 0.30+	Dedicated high-performance RAG & embedding microservice (Port 8001)
Vector Store	Chroma DB	0.5.5+	Persistent vector store with cosine matching & course metadata filters
Embedding Model	SentenceTransformers	all-MiniLM-L6-v2	384-dimensional dense vectors for semantic lecture retrieval
Speech-to-Text	Faster-Whisper	1.0+ (small/int8)	High-speed speech transcription for YouTube and uploaded video audio
LLM Inference	Groq Cloud API	gpt-oss-120b	Sub-second ultra-fast LLM generation for RAG synthesis & translation
Database	PostgreSQL / SQLite	16.x / 3.x	ACID compliant relational persistence with foreign keys & indexing
Cloud Storage	AWS S3 + boto3	1.35+	Private encrypted bucket with 1-hour presigned URL authorization

4. End-to-End System Architecture & Topology

LearnMate is built on a **Decoupled Microservice & Asynchronous Pipeline Architecture**. The system isolates high-throughput web traffic, persistent WebSocket connections, compute-heavy AI tasks, and transactional background queues into specialized, decoupled processes.



Key Architectural Invariants:

- Zero Direct Client Access to Vector DB / AI Engine:** The React client never contacts FastAPI (Port 8001) directly. All requests pass through Django DRF (`/api/ai/chat/`) which authenticates the JWT user and injects the shared `X-Internal-Secret` header.
- Non-Blocking Media Pipeline:** Audio extraction, Faster-Whisper transcription, and ChromaDB vector embedding run strictly inside background Celery worker threads. Mentors receive instantaneous HTTP 201 responses while lesson status transitions from `pending` → `processing` → `completed`.
- Course-Isolated Semantic Vector Spaces:** ChromaDB queries enforce a strict `where={'course\_id': course\_id}` filter, preventing context leaks across different courses.

5. Complete Project Directory Structure & File Manifest

The LearnMate codebase is organized into three clean root directories: `backend/`, `ai-service/`, and `frontend/`.

```
learnmate-2/
+-- backend/                                     # Django 6 & DRF Core Backend
|   +-- core/                                   # Root configuration package
|   |   +-- settings.py                       # Global settings (DB, JWT, AWS S3, Celery, Channels)
|   |   +-- urls.py                           # Root API routing (/api/accounts, /api/courses, etc.)
|   |   +-- asgi.py                           # Daphne ASGI router (HTTP + WebSocket JWTAuth)
|   |   +-- wsgi.py                           # Standard WSGI endpoint
|   |   +-- celery.py                         # Celery worker application initialization
|   +-- apps/
|   |   +-- accounts/                         # User entity, SimpleJWT auth, OTP emails, TOTP MFA
|   |   |   +-- models.py                     # User (AbstractUser), OTP models
|   |   |   +-- views.py                       # Register, SendOTP, VerifyOTP, Login, MFA views
|   |   |   +-- serializers.py                # User & Auth payload serializers
|   |   |   +-- urls.py                       # /api/accounts/*
|   |   +-- profiles/                         # Student & Mentor extended metadata
|   |   |   +-- models.py                     # StudentProfile, MentorProfile (OneToOne User)
|   |   |   +-- views.py & serializers.py
|   |   |   +-- urls.py                       # /api/profile/*
|   |   +-- courses/                          # Course catalog, syllabus, enrollments, progress
|   |   |   +-- models.py                     # Course, Module, Lesson, LessonResource, Enrollment, LessonProgress
|   |   |   +-- views.py                       # Mentor CRUD, Student Enrollment, Player, Statistics
|   |   |   +-- tasks.py                      # Celery tasks: send_mail, Whisper transcription, embed_lesson
|   |   |   +-- services/transcription.py      # yt-dlp + Faster-Whisper + Groq translation pipeline
|   |   |   +-- urls.py                       # /api/courses/*
|   |   +-- chat/                             # Real-time messaging & WebSocket consumers
|   |   |   +-- models.py                     # ChatRoom, Message, GroupChat, GroupMember, GroupMessage
|   |   |   +-- consumers.py                  # ChatConsumer (1-on-1), GroupConsumer (Batch WebSocket)
|   |   |   +-- middleware.py                 # JWTAuthMiddleware (WebSocket handshake auth)
|   |   |   +-- routing.py                    # ws/chat/<room_id>/ and ws/group/<group_id>/
|   |   |   +-- urls.py                       # /api/chat/*
|   |   +-- adminpanel/                       # Administrator analytics, user/course moderation
|   |   |   +-- views.py                       # dashboard.py, users.py, courses.py, mentors.py, reports.py
|   |   |   +-- urls.py                       # /api/adminpanel/*
|   |   +-- ai/                               # Django-to-FastAPI RAG gateway proxy
|   |   |   +-- views.py                       # AIChatView (Proxies RAG requests with secret header)
|   |   |   +-- urls.py                       # /api/ai/chat/
|   +-- manage.py
|   +-- requirements.txt
|   +-- .env                                   # Database, AWS S3, Brevo, Groq, Secret Keys
+-- ai-service/                                # Dedicated FastAPI RAG Microservice
|   +-- main.py                               # FastAPI routes (/embed, /lesson/{id}, /rag-chat, /health)
|   +-- services/
|   |   +-- embeddings.py                     # SentenceTransformers all-MiniLM-L6-v2 chunking & vectorization
|   |   +-- vectorstore.py                   # ChromaDB PersistentClient collection manager
|   |   +-- rag.py                            # Groq LLM grounded synthesis pipeline
|   +-- chroma_data/                          # Local persistent Chroma vector database directory
|   +-- requirements.txt
|   +-- .env                                   # GROQ_API_KEY, INTERNAL_API_SECRET
+-- frontend/                                  # React 19 Single Page Application
|   +-- package.json
|   +-- vite.config.js
|   +-- index.html
|   +-- src/
|   |   +-- App.jsx                           # Central route tree with ProtectedRoute RBAC guards
|   |   +-- api.js                            # Axios instance + 401 automatic JWT token refresh
|   |   +-- context/
|   |   |   +-- AuthContext.jsx               # Global user state, login, logout, profile loading
|   |   |   +-- ThemeContext.jsx              # Theme state provider
|   |   +-- components/
|   |   |   +-- ProtectedRoute.jsx            # Role-based route guard
|   |   |   +-- DashboardLayout.jsx           # Standard shell with Sidebar & Topbar
|   |   |   +-- Sidebar.jsx & Topbar.jsx      # Role-aware navigation & profile controls
|   |   +-- pages/
|   |   |   +-- auth/                         # Login.jsx, Register.jsx, ForgotPassword.jsx
|   |   |   +-- dashboards/
|   |   |   |   +-- student/                  # StudentDashboardOverview, StudentCourses, StudentLessonViewer...
|   |   |   |   +-- mentor/                  # MentorDashboardOverview, CourseSyllabusEditor, CourseStudents...
|   |   |   |   +-- admin/                   # AdminDashboardOverview, AdminCourses, AdminUsers, AdminReports...
```

6. Database Architecture & Relational ER Schema

LearnMate uses a relational PostgreSQL schema designed for strict referential integrity, relational cascades, and optimal query performance across course hierarchies, enrollments, and real-time messaging.

Model Name	Key Fields & Types	Relationships & Constraints
accounts.User (AbstractUser)	• id (UUIDField, PK, default=uuid4) • email (EmailField, unique=True, USERNAME_FIELD) • username (CharField, max_length=150) • role (admin mentor student, default=student) • is_verified (BooleanField, default=False) • mfa_enabled (BooleanField, default=False) • mfa_secret (CharField, max_length=255, nullable)	Central entity referenced across all apps. Unique constraint on email. Custom auth model configured in AUTH_USER_MODEL.
accounts.OTP	• user (ForeignKey → User, on_delete=CASCADE) • code (CharField, length=6) • otp_type (email_verification login password_reset) • is_used (BooleanField, default=False) • created_at (DateTimeField, auto_now_add=True)	Single-use numeric passcodes for email confirmation and 2-step logins. 5-minute validity window.
profiles.StudentProfile & MentorProfile	• StudentProfile: user (OneToOne), bio, grade, learning_goal • MentorProfile: user (OneToOne), specialization, experience (int)	1-to-1 extension tables linked directly to User.id with cascading deletion.
courses.Course	• id (AutoField, PK) • title (CharField, max 255), description (TextField) • thumbnail (ImageField → AWS S3 / course_thumbnails/) • mentor (ForeignKey → User, related_name='courses') • level (beginner intermediate advanced) • duration (CharField), status (draft published rejected)	Root course entity. Moderated by admins before transition to published.
courses.Module	• course (ForeignKey → Course, related_name='modules') • title (CharField), description (TextField) • order (PositiveIntegerField)	Hierarchical syllabus section. Ordered ascending by order.
courses.Lesson	• module (ForeignKey → Module, related_name='lessons') • title, description, duration, order • lesson_type (video pdf quiz assignment) • source_type (youtube upload) • video_url (URLField), video_file (FileField → S3) • is_preview (BooleanField, default=False) • transcript (TextField, English for RAG) • original_transcript (TextField, spoken language) • transcript_status (pending processing completed failed) • embedding_status (pending completed failed)	Core learning unit. Houses both video links/files, dual transcripts, and vector status flags.
courses.LessonResource	• lesson (ForeignKey → Lesson, related_name='resources') • title (CharField), resource_type (pdf doc img zip link) • file (FileField → AWS S3 / lesson_resources/) • external_url (URLField, nullable)	Attached study notes and code downloads. Protected by S3 presigned URLs.
courses.Enrollment & LessonProgress	• Enrollment: student (FK User), course (FK Course), is_completed, enrolled_at • LessonProgress: student (FK User), lesson (FK Lesson), is_completed, completed_at	Unique constraints: (student, course) and (student, lesson). Enforces single enrollment.
chat.ChatRoom, Message, GroupChat, GroupMessage	• ChatRoom: user1 (FK), user2 (FK) [unique_together] • Message: room (FK), sender (FK), receiver (FK), message, is_read • GroupChat: name, created_by (FK), group_type • GroupMember: group (FK), user (FK) • GroupMessage: group (FK), sender (FK), message	Relational persistence layer for all direct and group WebSocket chats.

7. Authentication, Multi-Factor Auth (MFA/TOTP) & Security

LearnMate enforces a multi-layered security model combining **Email OTP Verification**, **JSON Web Tokens (SimpleJWT)**, and **RFC 6238 TOTP Multi-Factor Authentication**.

7.1 Registration & Email Verification Sequence

1. User submits { username, email, password, role } to POST /api/accounts/register/.
2. Backend saves User record with is_verified=False and creates an OTP record (type='email_verification').
3. Celery / SMTP dispatches a 6-digit verification code to the user's email address.
4. User submits the code to POST /api/accounts/verify-otp/. Backend marks is_verified=True and invalidates the OTP.

7.2 Login Flow (2-Step Credential + MFA Verification)

1. User submits { email, password } to POST /api/accounts/login/.
2. If valid, backend generates an OTP and sends it via email, returning { message: 'OTP sent to email' }.
3. User inputs code to POST /api/accounts/verify-mfa/. Backend verifies the code and returns SimpleJWT token pair:

```
// Successful Login Response Payload:
{
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", // Valid for 1 Hour
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", // Valid for 7 Days
  "role": "student",
  "email": "student@example.com",
  "user_id": "8f14e45f-9a1b-4d78-b112-9c3f0b31e9c2"
}
```

7.3 Axios 401 Silent Token Refresh Interceptor (`frontend/src/api.js`)

The React frontend automatically attaches `Authorization: Bearer` to all outgoing REST calls. If a request receives an HTTP 401 response, Axios pauses outgoing traffic, posts the stored `refresh\_token` to /api/accounts/token/refresh/, updates `localStorage`, and retries the original request seamlessly without user disruption.

7.4 Authenticator App 2FA (RFC 6238 TOTP)

Users can enable Authenticator App 2FA (Google Authenticator, Authy, 1Password). The server generates a base32 cryptographic seed via `pyotp.random\_base32()`, stores it in `User.mfa\_secret`, and verifies 6-digit rolling codes via POST /api/accounts/verify-mfa-setup/.

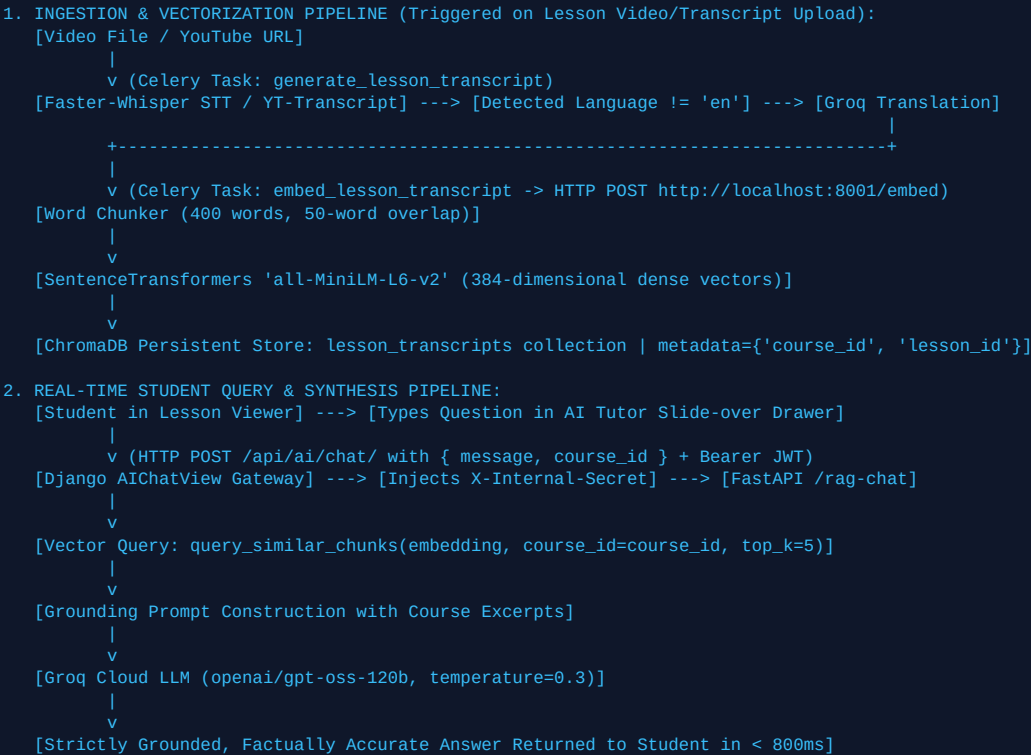
8. Backend Core Architecture (Django 6 & DRF)

The Django backend is modularized into specialized apps under `backend/apps/`. Each app contains dedicated `models.py`, `serializers.py`, `views.py`, `urls.py`, and where applicable, `tasks.py` and `consumers.py`.

9. AI Microservice & RAG Tutoring Pipeline (FastAPI + ChromaDB)

Located in `ai-service/`, the AI engine runs as an independent FastAPI microservice on **Port 8001**. It provides isolated, high-speed vector embeddings, persistent vector storage, and grounded RAG answer synthesis.

LEARNMATE RETRIEVAL-AUGMENTED GENERATION (RAG) PIPELINE



INTERNAL MICROSERVICE SECURITY

FastAPI endpoints (/embed, /lesson/{id}, /rag-chat) enforce strict header verification via `verify_internal_secret` dependency. Requests without a valid `X-Internal-Secret` header receive an immediate HTTP 401 Unauthorized response, ensuring external users cannot query or tamper with vector data directly.

10. Speech Transcription, Translation & Vector Embeddings

The transcription pipeline in `apps.courses.services.transcription`` provides bulletproof speech-to-text processing for both YouTube lectures and uploaded video files:

- YouTube Captions Check:** Attempts to fetch manual or auto-generated YouTube captions using ``youtube_transcript_api``. Validates speech density: if words-per-minute is less than 60 (sparse/broken captions), it discards them and falls back to Faster-Whisper.
- Whisper Audio Extraction:** Downloads the audio track using ``yt-dlp`` to a temporary directory and transcribes with ``faster_whisper.WhisperModel("small", device='cpu', compute_type='int8')``.
- Language Detection & Groq Translation:** Uses ``langdetect`` on the raw transcript. If the detected language is non-English (e.g. Malayalam, Spanish, Hindi), it sends the transcript to Groq LLM with a system prompt instructing exact pedagogical translation to English. Stores ``original_transcript`` and English ``transcript``.
- Automated Vectorization:** Dispatches ``embed_lesson_transcript.delay(lesson.id)`` to chunk and store embeddings in ChromaDB.

11. Real-time WebSockets Messaging (Channels + Redis)

Bidirectional real-time communication is powered by **Django Channels 4.0**, **Daphne ASGI**, and **Redis Channel Layer** (``channels_redis``).

Feature / Protocol	WebSocket Specification & Implementation Details
Endpoints	- Direct 1-on-1 Chat: <code>ws://<host>/ws/chat/<room_id>/?token=<jwt></code> - Batch Group Chat: <code>ws://<host>/ws/group/<group_id>/?token=<jwt></code>
Authentication	JWTAuthMiddleware parses the JWT access token from the WebSocket query string or headers, validates the signature via <code>UntypedToken</code> , and sets <code>scope['user']</code> .
Room Isolation	Each room binds to a deterministic Redis group (e.g. <code>chat_12</code> or <code>group_4</code>). Only verified participants can connect; unauthorized connection attempts are closed with code 4003.
Event Types	<code>message</code>: Persists to DB (Message model) and broadcasts <code>chat_message</code> payload. <code>typing</code>: Broadcasts real-time typing indicators to peer participants. <code>user_online</code>: Notifies subscribers when a peer connects.
Fallback & History	Historical messages are loaded via HTTP REST (<code>GET /api/chat/rooms/<id>/messages/</code>) upon room mount, followed by immediate WebSocket subscription for live updates.

12. Asynchronous Workers & Scheduling (Celery + Redis)

Celery executes background tasks asynchronously with Redis (`redis://127.0.0.1:6379/0`) acting as the message broker and result backend.

Task Name	Trigger / Schedule	Functional Responsibility
<code>generate_lesson_transcript</code>	Triggered on Lesson Create / Update	Downloads audio via yt-dlp, runs Faster-Whisper, translates non-English speech via Groq, saves transcripts, and triggers embedding.
<code>embed_lesson_transcript</code>	Triggered after transcript completion	Sends transcript text to FastAPI <code>/embed</code> with max 3 retries on transient connection failures.
<code>retry_stuck_embeddings</code>	Celery Beat cron (every 10 min)	Safety net: finds lessons whose transcript finished >10 min ago but embedding never completed, and re-queues them.
<code>send_inactivity_reminders</code>	Celery Beat cron (daily)	Identifies active student enrollments with >72h of inactivity and dispatches motivational reminder emails.
<code>send_course_completion_email</code>	Triggered on 100% course completion	Dispatches course completion congratulations email to the student.

13. Frontend Architecture & Client State (React 19 + Vite)

The React 19 frontend is structured around role-based modularity, global context providers, and responsive Tailwind v4 design:

- ``AuthContext.jsx``: Manages global authentication, JWT tokens in ``localStorage``, user role decoding, and fetches student/mentor profile details on initial boot.
- ``ProtectedRoute.jsx``: Evaluates ``user.role`` against ``allowedRoles`` (e.g. ``['student']``, ``['mentor']``, ``['admin']``). Redirects unauthenticated visitors to ``/login`` and role-mismatched users to their respective home dashboards.
- **Dashboard Layout**: Unified shell with responsive collapsible ``Sidebar.jsx``, active route indicators, and ``Topbar.jsx`` user status menu.

14. Cloud Media Storage & AWS S3 Signed URLs

All course thumbnails, uploaded lesson video files, and student downloadable resources are stored in an encrypted private AWS S3 bucket (learnmate-videos-storagebox). The bucket rejects all direct public HTTP reads. Django generates time-limited presigned URLs (1-hour validity via ``AWS_QUERYSTRING_EXPIRE = 3600``) when serialized for authenticated students and mentors.

15. Comprehensive HTTP REST API Reference

The Django REST Framework backend exposes clean RESTful endpoints under `/api/`:

Method & Endpoint	Auth & Role	Description & Request / Response Notes
POST /api/accounts/register/	Public	Registers new user (email, username, password, role). Dispatches email verification OTP.
POST /api/accounts/verify-otp/	Public	Verifies 6-digit email OTP. Sets <code>is_verified=True</code> .
POST /api/accounts/login/	Public	Validates credentials and dispatches login OTP to user email.
POST /api/accounts/verify-mfa/	Public	Verifies login OTP. Returns { access, refresh, role, email }.
POST /api/accounts/token/refresh/	Public	Rotates expired JWT access token using stored refresh token.
POST /api/accounts/forgot-password/	Public	Dispatches password reset OTP to user email.
POST /api/accounts/reset-password/	Public	Resets user password with valid OTP code.
GET /api/profile/student/	JWT (Student)	Returns student profile (bio, grade, learning goals).
GET /api/profile/mentor/	JWT (Mentor)	Returns mentor profile (specialization, experience).
GET /api/courses/	JWT	Lists all approved, published courses in the public catalog.
GET /api/courses/student/<id>/	JWT (Student)	Course details with modules, lessons, and enrollment status.
POST /api/courses/student/<id>/enroll/	JWT (Student)	Enrolls authenticated student in the specified course.
GET /api/courses/student/my-courses/	JWT (Student)	Lists enrolled courses with completion percentages.
POST /api/courses/lessons/<id>/complete/	JWT (Student)	Marks lesson complete. Dispatches completion email if 100%.
GET /api/courses/student/courses/<id>/progress/	JWT (Student)	Calculates percentage completed and completed lesson ID list.
POST /api/courses/create/	JWT (Mentor)	Creates a new draft course with title, description, level, thumbnail.
GET /api/courses/my-courses/	JWT (Mentor)	Lists all courses authored by the authenticated mentor.
POST /api/courses/modules/create/	JWT (Mentor)	Adds a module to a course syllabus.
POST /api/courses/lessons/create/	JWT (Mentor)	Adds a lesson (YouTube URL or file upload). Triggers Whisper STT task.
GET /api/courses/mentor/dashboard/	JWT (Mentor)	Mentor overview stats: total students, courses, active enrollments.
GET /api/courses/mentor/courses/<id>/students/	JWT (Mentor)	List of all enrolled students with individual progress percentages.
GET /api/adminpanel/admin/	JWT (Admin)	System-wide metrics: total users, courses, active enrollments.
GET /api/adminpanel/courses/pending/	JWT (Admin)	Lists courses waiting for publication approval.
POST /api/adminpanel/courses/<id>/publish/	JWT (Admin)	Approves and publishes course to public student catalog.
GET /api/adminpanel/users/	JWT (Admin)	List, filter, and moderate platform users.
POST /api/ai/chat/	JWT	Payload: { message, course_id }. Proxies query to FastAPI RAG service.
GET /api/chat/rooms/	JWT	Lists user's 1-on-1 direct message conversation rooms.
GET /api/chat/rooms/<id>/messages/	JWT	Retrieves historical messages for direct chat room.
GET /api/chat/groups/	JWT	Lists user's cohort group chat channels.

16. Environment Configuration & Secret Management

The following configuration files must be established in their respective root directories:

16.1 Backend Environment Configuration (`backend/.env`)

```
# Database Credentials (PostgreSQL)
DB_NAME=learnmate_db
DB_USER=postgres
DB_PASSWORD=your_secure_db_password
DB_HOST=localhost
DB_PORT=5432

# Django Security & Settings
SECRET_KEY=django-insecure-your-production-secret-key-here
DEBUG=False

# Transactional SMTP Email Service (Brevo / SendGrid)
EMAIL_HOST=smtp-relay.brevo.com
EMAIL_PORT=587
EMAIL_HOST_USER=your_brevo_smtp_user@smtp-brevo.com
EMAIL_HOST_PASSWORD=xsmtpsib-your-smtp-master-key
EMAIL_USE_TLS=True
DEFAULT_FROM_EMAIL=no-reply@learnmate.com

# AI Microservice & LLM API Keys
GROQ_API_KEY=gsk_your_groq_production_api_key
INTERNAL_API_SECRET=e288669e46a85594a53b62dd5ecc986822de12499d9acc5d7d034391e3e47b29

# AWS S3 Encrypted Media Bucket
AWS_ACCESS_KEY_ID=AKIA_YOUR_AWS_KEY_ID
AWS_SECRET_ACCESS_KEY=your_aws_secret_access_key
AWS_STORAGE_BUCKET_NAME=learnmate-videos-storagebox
AWS_S3_REGION_NAME=us-east-1
```

16.2 AI Service Environment Configuration (`ai-service/.env`)

```
GROQ_API_KEY=gsk_your_groq_production_api_key
INTERNAL_API_SECRET=e288669e46a85594a53b62dd5ecc986822de12499d9acc5d7d034391e3e47b29
```

17. Production Deployment, Nginx, Systemd, Domain & SSL (HTTPS)

For production hosting on Ubuntu Linux (e.g. AWS EC2, DigitalOcean, or Linode), the architecture runs Daphne ASGI, FastAPI, Celery, and Redis behind an Nginx reverse proxy with automated Let's Encrypt SSL.

17.1 Production Nginx Configuration (`/etc/nginx/sites-available/learnmate`)

```
server {
    server_name api.learnmate.com learnmate.com;

    # Frontend Production Build (Static React Files)
    location / {
        root /var/www/learnmate/frontend/dist;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    # Django HTTP REST APIs
    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Django Channels WebSocket Connections
    location /ws/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "Upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 86400;
    }
}
```

17.2 Domain & Automated Let's Encrypt SSL Setup

```
# 1. Install Certbot for Nginx
sudo apt update && sudo apt install -y certbot python3-certbot-nginx

# 2. Request and install SSL certificate for domains
sudo certbot --nginx -d learnmate.com -d api.learnmate.com

# 3. Test automatic certificate renewal
sudo certbot renew --dry-run
```

17.3 Systemd Services Architecture

Four systemd service units manage background processes: • **`learnmate-daphne.service`**: Runs ``daphne -b 127.0.0.1 -p 8000 core.asgi:application`` • **`learnmate-ai.service`**: Runs ``uvicorn main:app --host 127.0.0.1 --port 8001`` in ``ai-service/`` • **`learnmate-celery.service`**: Runs ``celery -A core worker --loglevel=info --concurrency=4`` in ``backend/`` • **`learnmate-beat.service`**: Runs ``celery -A core beat --loglevel=info`` in ``backend/``

18. Testing, Verification, QA & Health Checks

A complete verification checklist to validate system health across all layers:

Subsystem	Verification Command / Action	Expected Output / Behavior
Backend Health	<code>python manage.py check</code> <code>python manage.py test</code>	System check identified no issues (0 silenced).
AI Service Health	<code>curl -i http://127.0.0.1:8001/health</code>	HTTP/1.1 200 OK → {"status":"ok"}
Redis Broker	<code>redis-cli ping</code>	PONG
Celery Tasks	<code>celery -A core inspect active</code>	Lists running Celery worker nodes without exceptions.
Frontend Build	<code>npm run build in frontend/</code>	Zero TypeScript / JSX errors; builds optimized static bundle.

19. Troubleshooting Manual & Common Failure Modes

Symptom / Issue	Root Cause	Remediation Step
WebSocket connects then immediately closes (4003)	Invalid, expired, or missing JWT token during WS handshake.	Ensure React passes <code>?token=<access_token></code> in WS URL or refreshes expired token first.
AI Tutor returns 'AI service unavailable' (503)	FastAPI service on Port 8001 is down or INTERNAL_API_SECRET mismatch.	Verify FastAPI is running (<code>curl http://127.0.0.1:8001/health</code>) and secrets in both <code>.env</code> files match.
Lesson stays stuck on transcript_status='pending'	Celery worker process is not active or crashed during Whisper processing.	Start Celery worker: <code>celery -A core worker --loglevel=info</code> . Check Celery logs.
S3 Video / Resource returns 403 Forbidden	Presigned URL expired (>1h) or incorrect AWS S3 credentials.	Verify AWS credentials in <code>backend/.env</code> and ensure media URLs are freshly serialized.
CORS errors on frontend REST API calls	Missing corsheaders in Django settings or unallowed origin.	Verify <code>CORS_ALLOW_ALL_ORIGINS = True</code> or add origin to <code>CORS_ALLOWED_ORIGINS</code> .

20. Developer Onboarding: Rapid Codebase Bootstrap Guide

Welcome to the LearnMate engineering team! This final section is your step-by-step onboarding guide to bootstrap your local development environment in under 10 minutes.

Step 1: Clone Repository & Configure Prerequisites

Ensure you have **Python 3.10+**, **Node.js 18+**, **PostgreSQL**, and **Redis Server** installed on your machine.

Step 2: Bootstrap Django Backend

```
# 1. Navigate to backend directory and create virtual environment
cd backend
python -m venv venv
source venv/bin/activate    # On Windows: venv\Scripts\activate

# 2. Install dependencies
pip install -r requirements.txt

# 3. Configure backend/.env (copy sample values from Section 16)
# 4. Apply database migrations & create superuser
python manage.py migrate
python manage.py createsuperuser

# 5. Run Daphne ASGI development server (Port 8000)
python manage.py runserver
```

Step 3: Launch Background Workers (Redis & Celery)

```
# In a separate terminal tab: Start Redis Server
redis-server

# In another terminal tab (with backend venv activated):
celery -A core worker --loglevel=info --concurrency=2

# In another terminal tab (for periodic cron schedulers):
celery -A core beat --loglevel=info
```

Step 4: Bootstrap FastAPI AI & RAG Microservice

```
cd ai-service
python -m venv venv
source venv/bin/activate    # On Windows: venv\Scripts\activate
pip install -r requirements.txt

# Launch FastAPI microservice on Port 8001
uvicorn main:app --port 8001 --reload
```

Step 5: Bootstrap React Frontend

```
cd frontend
npm install

# Launch Vite development server on Port 5173
npm run dev
```

DEVELOPER CHEAT SHEET: KEY CODE LOCATIONS

- **Authentication Logic:** backend/apps/accounts/views.py
- **Syllabus & Video Ingestion:** backend/apps/courses/views.py & tasks.py
- **Speech Transcription & Translation:** backend/apps/courses/services/transcription.py
- **RAG Chunking & Embeddings:** ai-service/services/embeddings.py & vectorstore.py
- **WebSocket Consumers:** backend/apps/chat/consumers.py
- **Frontend Router & Guards:** frontend/src/App.jsx & components/ProtectedRoute.jsx
- **Axios Interceptor:** frontend/src/api.js